BUT SD

SAE Apprentissage pour l'IA

Cours n°3

Deep Learning & Images

Marc Métivier

IUT de Paris - Rives de Seine
Université Paris Cité

Plan du cours

1. Quelques solutions avancées

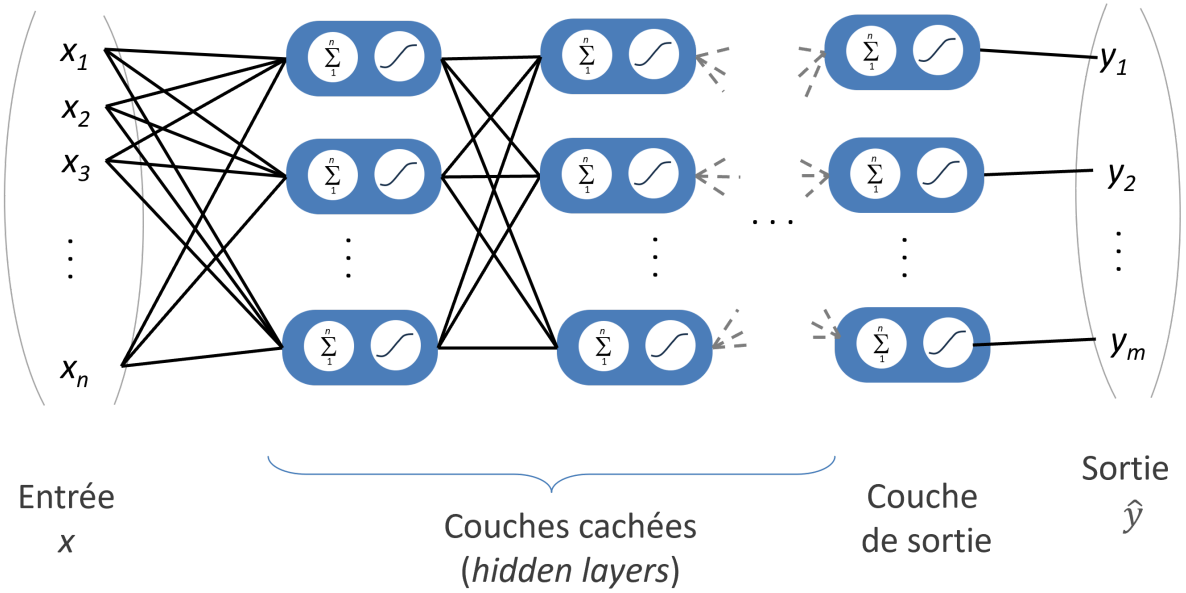
- Le coût pour les variables catégorielles
- Problèmes typiques du deep learning
- Descentes de gradient améliorées
- Méthodes diverses

2. Architectures pour le traitement des images

- Convolutions
- Réseaux de neurones convolutifs
- Architectures classiques

Réseaux de neurones

Le Percetron multi-couches / Multi-Layered Perceptron (MLP)



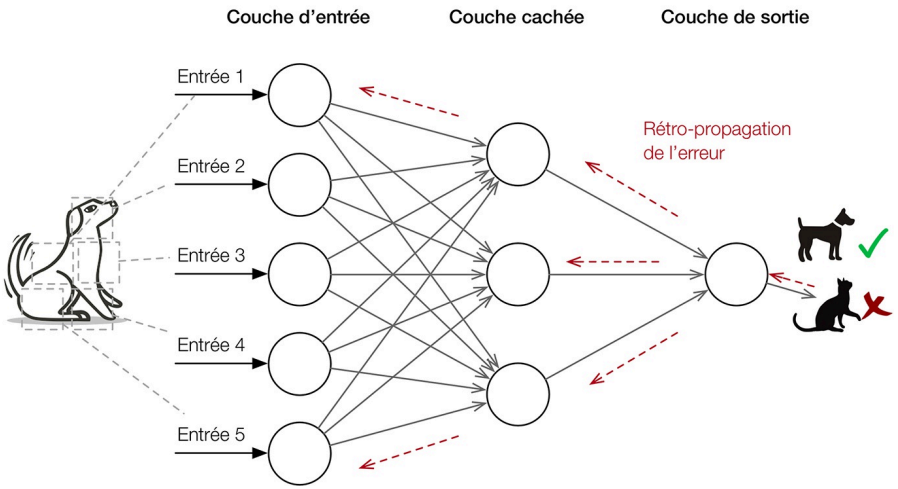
Un approximateur universelle (Cybenko 1989)

- **Toute fonction** peut être approchée, ***avec la précision que l'on veut***, par un MLP à une couche cachée ayant ***suffisemment*** de neurones.
- En pratique, le nbre de neurones de la couche cachée peut être très grand.

Apprentissage

Entraîner un RN = Mettre à jour les poids des connexions entre neurones

Algorithme *Backprop* : la rétro-propagation du gradient de l'erreur



D Cardon, JP Cointet, A Mazieres (2018), La revanche des neurones

- On calcule une **fonction de coût** (***loss***) basée sur l'erreur de prédiction
- On rétro-propage une correction basée sur le gradient de cette fonction coût

Les 3 problèmes supervisés

Régression

- ***Pas d'activation***
- Coût : ***Mean squared error (MSE)***

$$\ell(y, \hat{y}) = ||y - \hat{y}||_2^2 = \sum_{i=1}^{m} (y_i - \hat{y}_i)^2$$

Classification binaire

- Activation : ***Sigmoid*** (probabilité d'être dans la 1ère classe)
- Coût : ***Binary cross entropy***

$$\ell(y, \hat{y}) = -\frac{1}{m} \sum_{i=1}^{m} y_i . \log \hat{y}_i + (1 - y_i) . \log(1 - \hat{y}_i)$$

Classification multi-classe

- Activation : ***Softmax*** (distibution des probabilités d'être dans chaque classe)
- Coût : ***Categorical cross entropy***

$$\ell(y, \hat{y}) = -\sum_{i=1}^{m} y_i . \log \hat{y}_i$$

Coût & Variables catégorielles

Variable catégorielle ou "qualitative"

- Une valeur parmi un nombre limité de valeurs possibles

Représentation typique : le One Hot Encoding (OHE)

- Encodage "1 parmi n"
- Si $n = 8$ est le nombre de classes, appartenir à la 4ème classe est codée :

$$00010000$$

  - C'est la valeur cible des probabilités pour une donnée à classer dans la classe n°4
- Lorsque n est très grand : représentation "sparse" du One Hot Encoding
  - Spécifier uniquement l'entier correspondant au rang du 1 dans la représentation
  - La représentation "sparse" de 00010000 est 3 (car le premier rang est numéroté 0)

Calcul du coût (loss) avec Keras

- Valeur cible représentée en OHE : `categorical_entropy`
- Valeur cible représentée par un entier : `sparse_categorical_entropy`

Problèmes typiques

La disparition du gradient

- Le gradient devient trop faible au fur-et-à-mesure de la rétro-propagation
- Les premières couches apprennent très lentement
- Une configuration pouvant mener à ce genre situation est l'usage d'activations de type Sigmoïde ou Tanh avec un grand nombre de couches de neurones
    - Les dérivées sont faibles et provoquent des mises à jour de plus en plus petites de couche en couche
- Solution classique : utiliser l'activation ReLU

Problèmes typiques

Instabilité de l'apprentissage

- Les mises à jour des poids provoquent des oscillations
- Les activations sont instables au cours du temps
- Possible phénomène de l'"explosion de gradient"
- Quelques approches :
  - Utiliser un calcul de gradient avec "momentum" (cf. plus loin)
  - Ajouter une Batch-normalization
    - Elle consiste normaliser les activations d'une couche de neurones
    - Sur keras, il suffit d'ajouter une couche `BatchNormalization`

Descente de gradient avancée

Quelques méthodes "améliorées" :

- Adagrad
  - Adaptation du taux d'apprentissage en fonction de la fréquence des données
- Adadelta
  - Adagrad avec un taux d'apprentissage déterminé automatiquement
- RMSprop
  - Assez similaire à Adadelta
  - L'auteur (Hinton) conseille les paramètres suivants : $g = 0.9, h = 0.001$
- Adam (Adaptive Moment Estimation)
  - Mêmes avantages que Adadelta/RMSprop et momentum
  - Paramètres typiques : $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

La méthode du moment (*momentum*)

- La mise à jour des paramètres prend en compte les gradients passés
- Une moyenne pondérée entre le gradient immédiat et le gradient précédent
- Inspiré du calcul du moment en physique : le gradient agit comme une force

Problèmes typiques

Le sur-apprentissage

- **Symptôme** :
  - Le coût d'entraînement baisse alors que le coût d'évaluation stagne ou augmente
- **Le problème** :
  - Le modèle apprend ***trop bien*** les données d'entraînement
  - Il peut mémoriser les exemples au lieu d'apprendre des règles générales
- **Quelques approches pour le réseaux de neurones** :
  - Réduire la capacité : diminuer le nb de neurones, voire le nb de couches
  - Régularisation L1 ou L2 : pénaliser les poids pour éviter qu'ils soient trop élevés
  - Dropout
  - Data augmentation
  - Early stopping

Data Augmentation

- **Principe** : créer de nouvelles données à partir des données existantes
  - Certaines modifications peuvent parfois se faire sans altérer le sens des données
  - Par exemple dans les images :
    - un chat reste un chat s'il est retourné horizontalement
    - un avion déplacé dans l'image reste un avion
- Tout dépend des données
  - Retourner horizontalement le chiffre 3 fait perdre son sens
- Keras propose des ***couches d'augmentation*** des données selon le type transformation souhaité
  - Exemples : `RandomFlip("horizontal")`, `RandomRotation(0.1)`, `RandomZoom(0.1)`, ...

Dropout

- **Principe** : à chaque itération, on "éteint" aléatoirement certains neurones
  - Généralement utilisé dans les couches denses
  - L'idée est de désactiver temporairement des neurones lors de l'apprentissage
  - Le but est de perturber les caractéristiques apprises pour éviter le sur-apprentissage
- La contrepartie est une temps d'optimisation augmenté
- Keras propose une ***couche*** `Dropout` à ajouter au modèle
  - Il faut préciser un taux de désactivation (le taux de neurones à désactiver)

⚠ À désactiver en phase d'évaluation

Early Stopping

- **Principe** : arrêter l'entraînement lorsque le ***coût de test cesse de s'améliorer***
  - Arrêter l'apprentissage avant d'obtenir du sur-apprentissage
  - Réduit le temps de calcul
  - On peut mettre un nombre d'itérations importants et laisser le système s'arrêter

- Si le coût ne s'améliore pas pendant N époques consécutives → arrêt

  - N est appelé le paramètre de ***Patience***

- Dans Keras, on définit une fonction de type EarlyStopping pouvant être exécutée pendant l'entraînement (callback)

```
from keras.callbacks import EarlyStopping

early_stopping = EarlyStopping(monitor='val_loss',
    patience=5,restore_best_weights=True
)

model.fit(x_train, y_train,
    validation_data=(x_val, y_val),
    epochs=100,
    callbacks=[early_stopping]
)
```

Réseaux de neurones pour les images

Traitement des images

Problème du MLP pour le traitement des images

- Très grands nombres de paramètres

  Ex : pour une seule couche de 1000 neurones

  - MNIST : 28x28, 8bits (0.0008 M pixels) $\longrightarrow$ 785K paramètres
  - Image courante : 24 M pixels, (r, v, b) 3x8 bits $\longrightarrow$ 72G paramètres

- La cohérence spatiale spatiale est perdue par l'applatissement des données

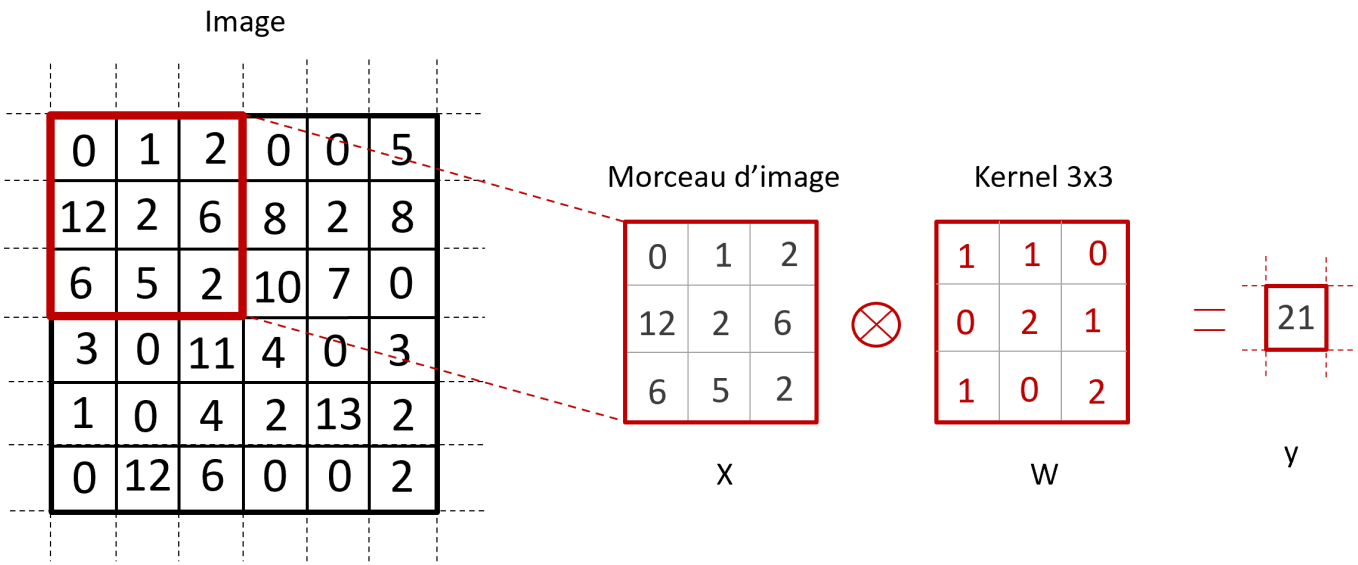
  - proximités des pixels, formes, ...

La solution : les convolutions 2D

- Utiliser des filtres que l'on peut déplacer sur l'image au lieu de tout connecter

Filtrage numérique par convolution

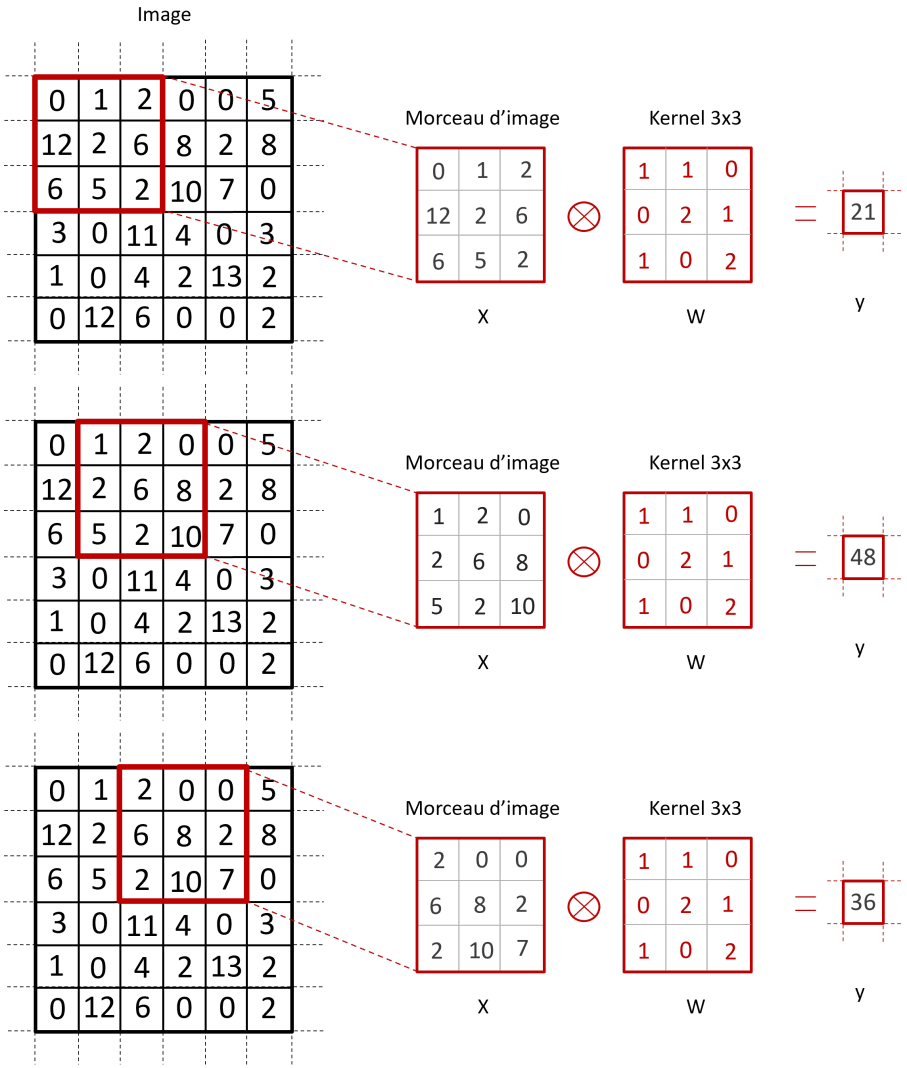
L'idée est d'appliquer des filtres sur des morceaux d'images



- Un filtre est défini par un noyau (kernel) et s'applique sur des morceaux d'image
- Il produit une valeur pour chaque morceau :

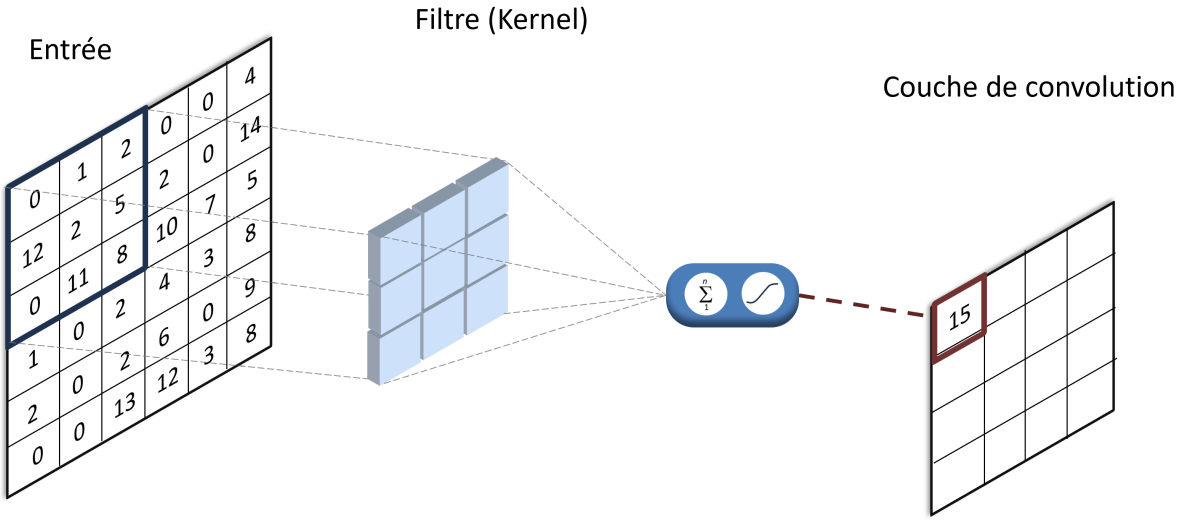
$$y = \sum_{i,j} x_{i,j} . w_{i,j}$$

Filtrage numérique par convolution



Couche de convolution

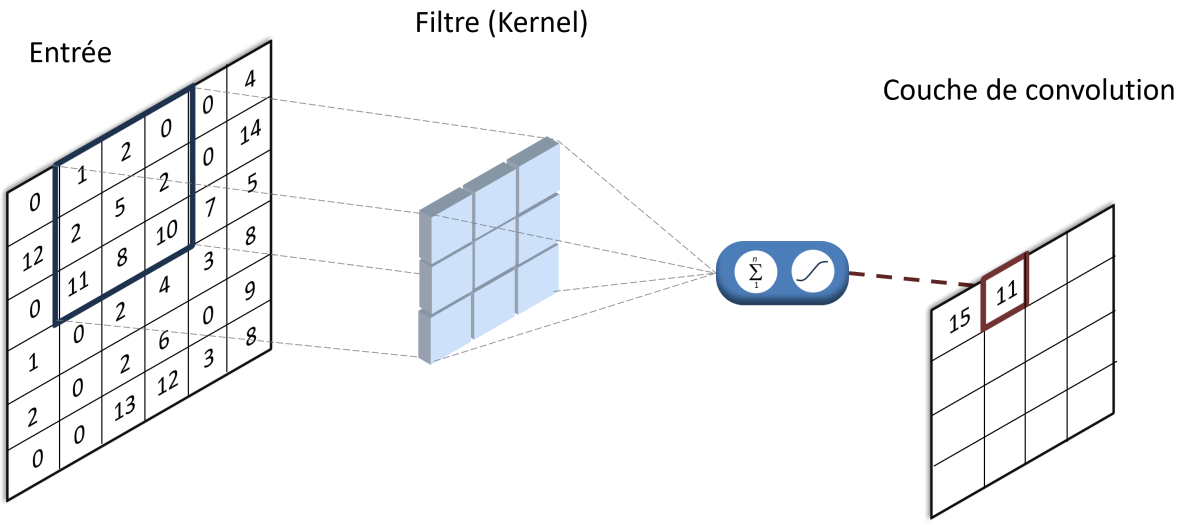
Chaque neurone est associé à un filtre



- Le filtre est appliqué sur les données d'entrée
- Les poids du filtres sont les poids du neurone (ici 3x3=9 paramètres)
- La fonction d'activation est typiquement ReLU

Couche de convolution

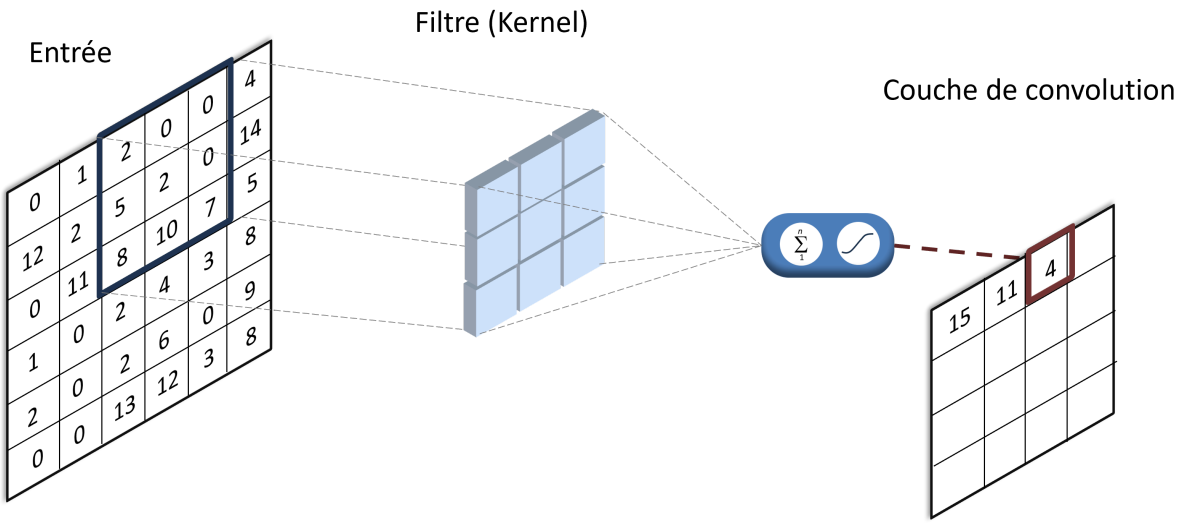
Chaque neurone est associé à un filtre



- Le filtre est déplacé sur les données d'entrée
- Une valeur de sortie est produite pour chaque morceau d'entrées

Couche de convolution

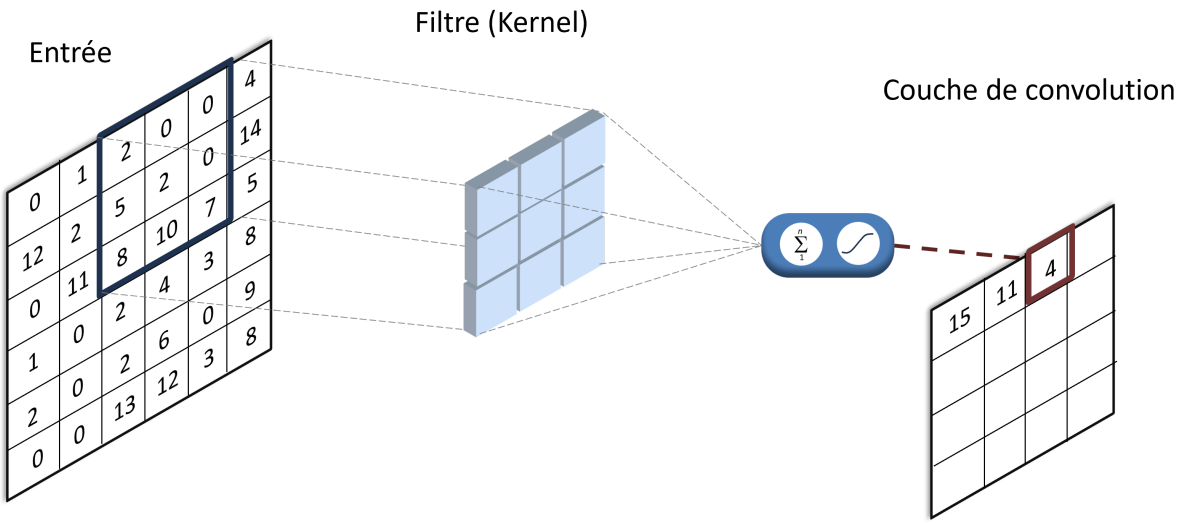
Chaque neurone est associé à un filtre



- Le filtre est déplacé sur les données d'entrée
- Une valeur de sortie est produite pour chaque morceau d'entrées

Couche de convolution

Chaque neurone est associé à un filtre



- Le filtre est déplacé sur les données d'entrée
- Une valeur de sortie est produite pour chaque morceau d'entrées

➡ Un seul neurone, avec quelques poids, produit une nouvelle table de valeurs

Prise en compte de la couleur

Code RVB : Rouge-Vert-Bleu

- Une couleur est la combinaison des intensités de rouge, de vert et de bleu
- Si chaque intensité est codée sur 8 bits (de 0 à 255) : +16M de couleurs
- Exemples :
  - (0, 0, 0) code le noir
  - (255, 255, 255) code le blanc
  - (100, 100, 100) code un gris
  - (255, 0, 0) code le rouge

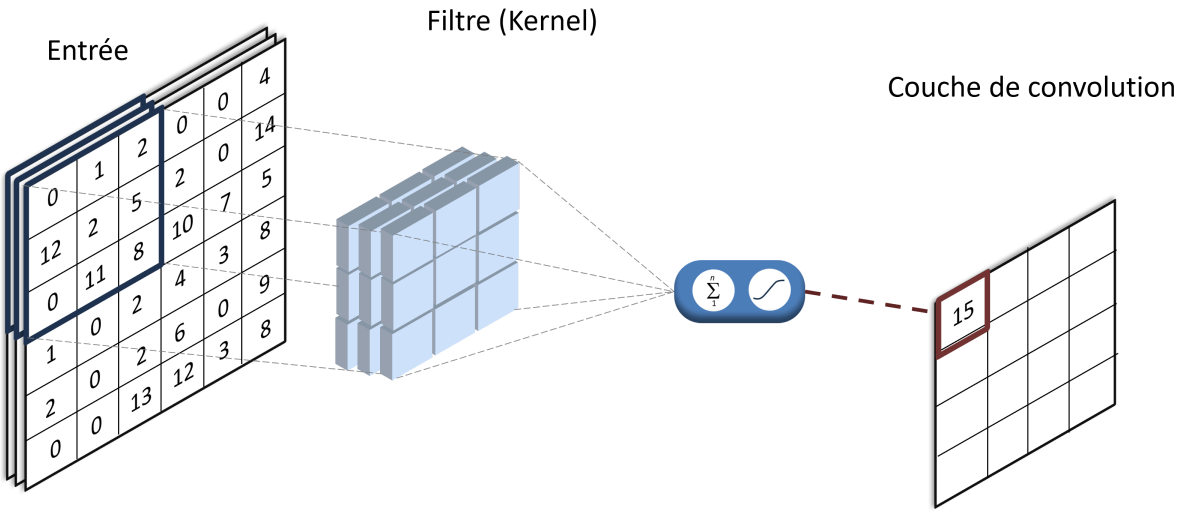
Codage des images couleurs

- Chaque image est composée de 3 canaux : un pour chaque couleur primaire
- Chaque canal est un image monochrome dans une couleur primaire
  - Il donne l'intensité de chaque pixel dans cette couleur primaire
- La superposition des canaux donne l'image en couleur

Couleurs et convolutions

Le fitre devient 3D

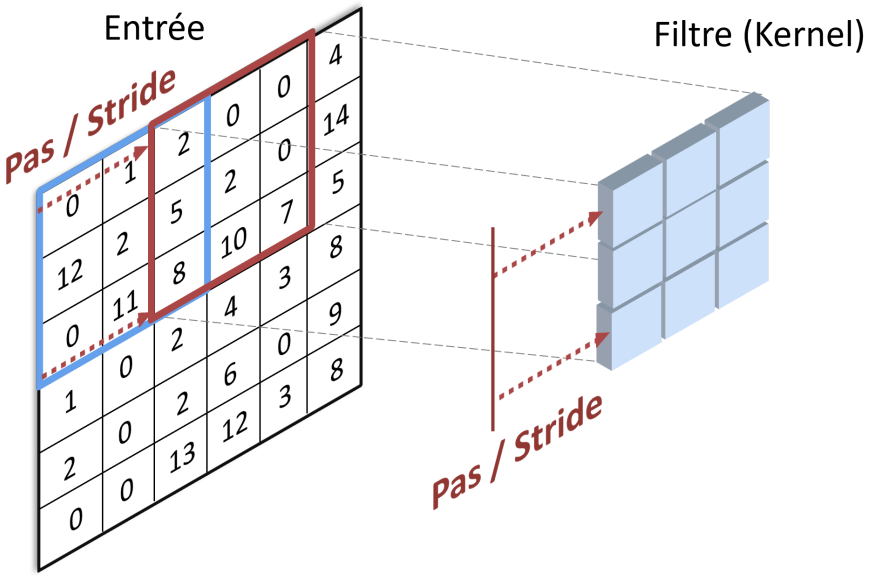
Chaque neurone applique un filtre avec 3 canaux



Le Pas / Stride

Le pas (stride) définit la manière dont est déplacé le filtre

- Par défaut, le pas est de 1 (cas des exemples précédents)
- Avec un pas de 2 : le filtre est déplacé de 2 au lieu de 1

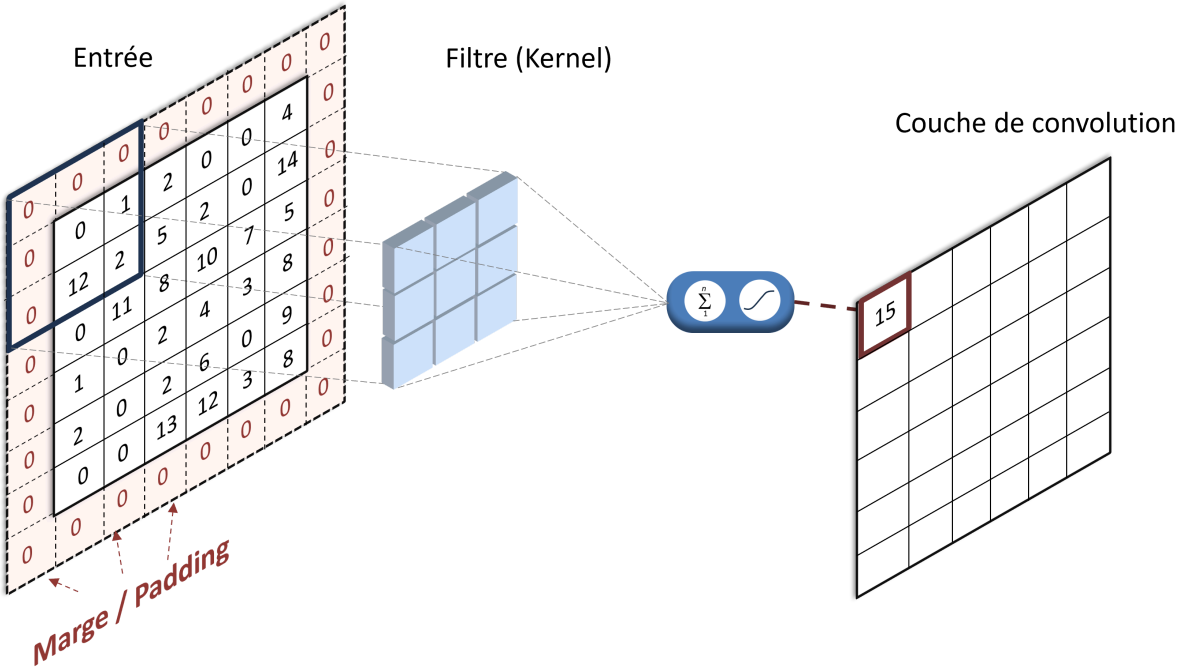


Ajout d'une Marge / Padding

La marge (padding) padding définit des valeurs pour déborder de l'image

Le cas classique : le zero-padding

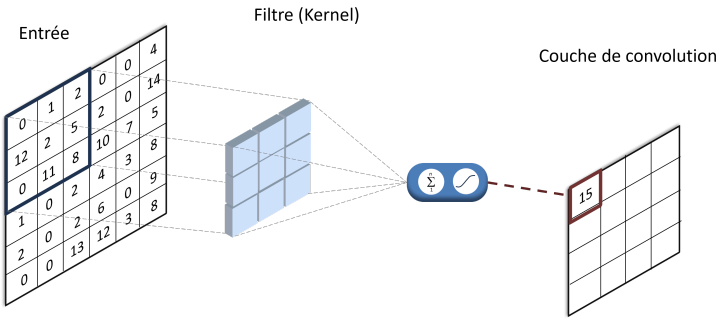
- Ajouter des 0 autour de l'image afin de pouvoir centrer le filtre sur les coins
- L'intérêt est d'obtenir un résultat de même taille que l'entrée



Avec ou sans padding

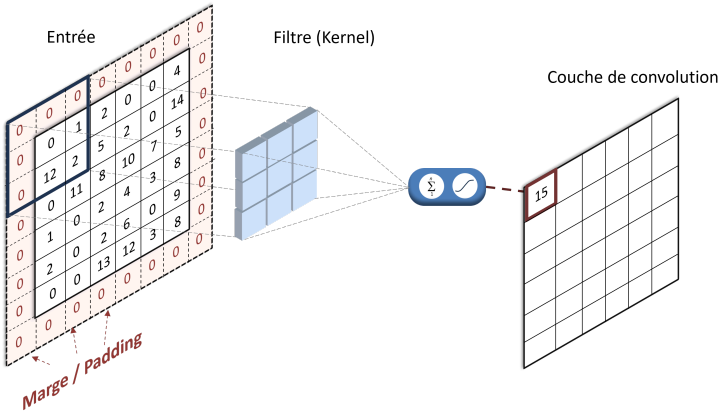
Sans marge :

- Sortie **plus petite que l'entrée**



Zero-padding de largeur adaptée :

- Les éléments de la marge valent 0
- Sortie **de même taille que l'entrée**
- Conditions :
  - Le filtre doit avoir un "centre" (dimensions impaires)
  - La largeur de marge doit être la moitié de la dimension du filtre



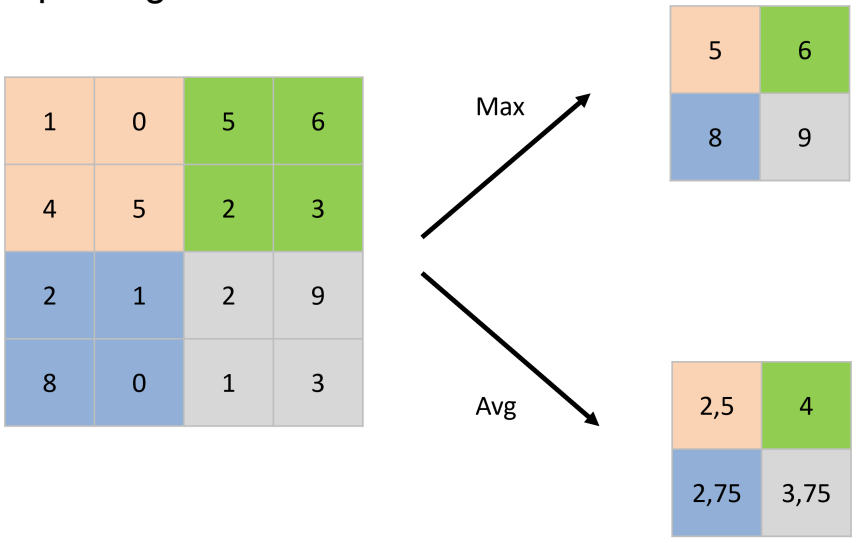
Couche de Pooling

Pooling : la "mise en commun"

- ***Compresser*** l'information de plusieurs valeurs de sortie en une seule valeur
- Permet de réduire la dimension de sortie
- Elle rend le réseau moins sensible aux translations éventuelles de l'image

Deux types classiques de pooling : le maximum ou la moyenne

- Exemple avec un pooling 2x2 :



Réseaux neuronaux convolutifs

Convolutional Neural Networks (CNN)

- Un réseau constitué de nombreuses couches de convolutions
- Chaque couche contient plusieurs neurones
- Chaque neurone produit un nouveau "plan de données"
- Introduits par Yann LeCun en 1989

Ils ont révolutionné l'analyse d'images

- La structure spatiale dans les données est conservée
- Moins de paramètres à apprendre
- Invariant par rapport à la position des objets et aux distortions dans la scène

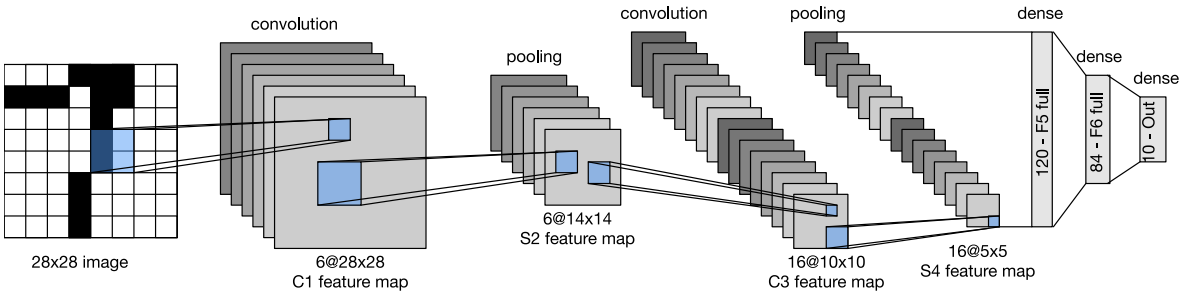
Architecture moderne typique :

- Partie extraction d'attributs (features extractor)
  - Empilement de séries de couches (Conv + ReLU + Max-pool)
- Partie classication
  - Empilement de couches (Dense + ReLU), puis Softmax en sortie

LeNet-5 (LeCun et al., 1998)

Une architecture de référence

- La première version (LeNet-1) date de 1989
- L'optimisation avaient pris plusieurs semaines sur les données MNIST



Source : https://d2l.ai/chapter_convolutional-neural-networks/lenet.html

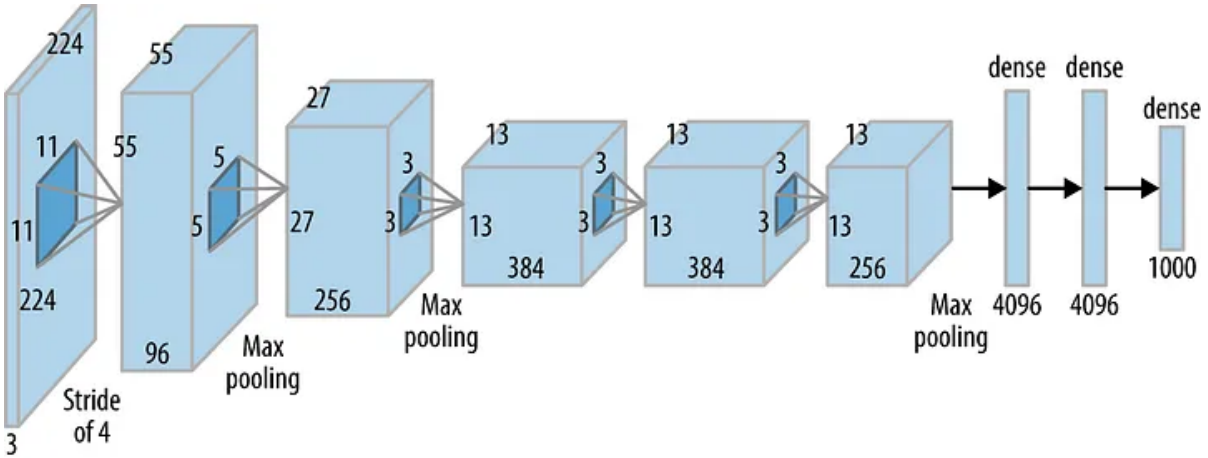
Quelques détails :

- Filtres de taille 5x5 et zero-padding de taille 2 sur la première couche
- Activations ***Tanh*** pour toutes les couches sauf la sortie ***Sigmoid***
- Pooling 2x2 utilisant la moyenne (avg-pooling)

AlexNet (Krizhevsky et al., 2012)

La vraie révolution des CNN

- CNN à 8 couches qui a gagné le challenge ILSVRC en 2012
- Particularités :
  - Des filtres de taille 11X11 puis 5x5 puis 3x3 et zero-padding
  - Activations ReLU pour éviter la disparition du gradient
  - Softmax en sortie
  - Max-Pooling
  - Data augmentation et dropout pour réduire le sur-apprentissage



Source : oreilly.com

Comparaison

LeNet

Image: 28 (height) × 28 (width) × 1 (channel)
↓
Convolution with 5×5 kernel+2 padding:28×28×6
↓ sigmoid
Pool with 2×2 average kernel+2 stride: 14×14×6
↓
Convolution with 5×5 kernel (no pad): 10×10×16
↓ sigmoid
Pool with 2×2 average kernel+2 stride: 5×5×16
↓ flatten
Dense: 120 fully connected neurons
↓ sigmoid
Dense: 84 fully connected neurons
↓ sigmoid
Dense: 10 fully connected neurons
↓
Output: 1 of 10 classes

AlexNet

Image: 224 (height) × 224 (width) × 3 (channels)
↓
Convolution with 11×11 kernel+4 stride:54×54×96
↓ ReLu
Pool with 3×3 max. kernel+2 stride: 26×26×96
↓
Convolution with 5×5 kernel+2 pad:26×26×256
↓ ReLu
Pool with 3×3 max. kernel+2 stride: 12×12×256
↓
Convolution with 3×3 kernel+1 pad:12×12×384
↓ ReLu
Convolution with 3×3 kernel+1 pad:12×12×384
↓ ReLu
Convolution with 3×3 kernel+1 pad:12×12×256
↓ ReLu
Pool with 3×3 max. kernel+2 stride: 5×5×256
↓ flatten
Dense: 4096 fully connected neurons
↓ ReLu, dropout p=0.5
Dense: 4096 fully connected neurons
↓ ReLu, dropout p=0.5
Dense: 1000 fully connected neurons
↓
Output: 1 of 1000 classes

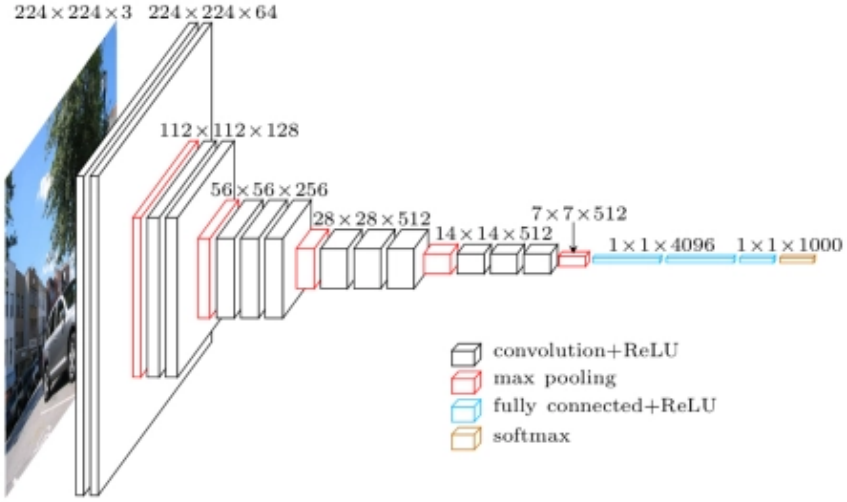
Source : wikipedia

VGGNet (2014)

Particularité

- VGG = "Visual Geometry Group" (un groupe de l'Université d'Oxford)
- Tous les filtres sont de taille 3x3
- Plusieurs couches de convolution avant le max-pooling

VGG16 : une architecture à 16 couches (13 conv, 3 denses)



Simonyan & Zisserman, 2015

VGG19 : une version de VGG à 19 couches, vainqueur de ILSVRC en 2014

Connexions résiduelles

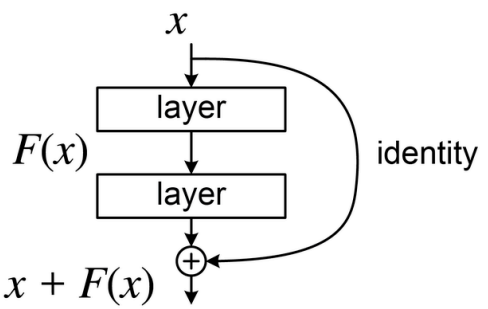
Un constat : ajouter une couche de neurones peut détériorer les performances

- Disparition du gradient
- Sur-apprentissage plus fréquemt
- Les réseaux profonds ont tendance à modéliser des fonctions trop compliquées
- Pourtant, une couche inutile pourrait "apprendre à ne rien faire"...

...mais en pratique ce n'est pas le cas !

Connexion résiduelle : un raccourci pour permettre plus de profondeur

- **Bloc résiduel** : les entrées ajoutées en sortie après deux couches du réseau

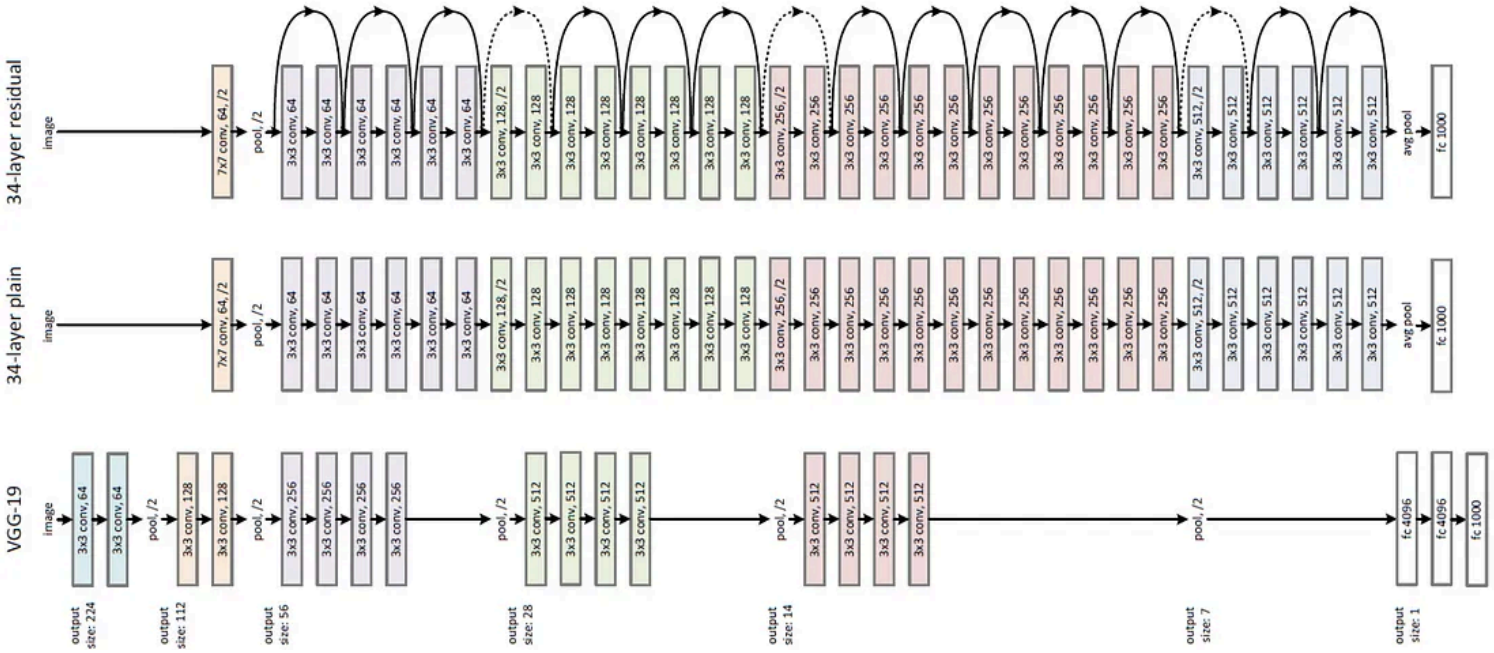


LunarLullaby, CC BY-SA 4.0, via Wikimedia Commons

- L'idée : si une couche est peu utile, le réseau apprend à privilégier le raccourci

ResNet (He et al., 2015)

Un empilement de blocs résiduels (tour résiduelle)



Source : (He et al., 2015)

- La tour résiduelle peut monter à beaucoup plus que 34 couches
- ResNet-152 (152 couches) est vainqueur de ILSVRC en 2015

Résumé des architectures

Réseau	Année	Particularité	top5 accuracy	Paramètres	FLOP
AlexNet	2012	Profondeur augmentée	84.70%	62M	1.5B
VGGNet	2014	Kernels de taille fixe	92.30%	138M	19.6B
ResNet-152	2015	Connexions résiduelles	95.51%	60.3M	11B

Source : https://towardsdatascience.com/the-w3h-of-alexnet-vggnet-resnet-and-inception-7baaaecccc96

Keras propose des architectures pré-entraînées sur ImageNet

- Voir : https://keras.io/api/applications/
- On y trouve VGG16, VGG19, ResNet-50, ResNet-152, etc.